

UMassAmherst

Manning College of Information
& Computer Sciences

Lab 1: Environment Setup & HOFs

Wednesday, September 9th, 2026



Weekly Lab Agenda

- Go over reminders/goals
- Review recent material
- Work in groups of 3-4 to solve a few exercises
 - Discussion leaders will assign groups today.
 - Groups will be remade every few weeks.
- Discussion leaders will walk around and answer questions.
- Solutions to exercises will be reviewed as a class.

Lab Policy

- Lab grade will be 50% attendance, 50% Gradescope submission
 - Must attend lab to receive submission credit
- Two lowest lab scores dropped
- Excusal form on Canvas for absences with documented reasons
 - Fill out before lab time (if not possible, then ASAP and within a week)
- You are expected to arrive on time and stay for the entire lab.
 - If you arrive late, you may be asked to attend a later lab section.

Reminders

- Please set up your development environment as soon as possible.
 - Use this lab time to get help with setup.
- Homework 1 is due next Monday (9/14) at 11:59pm.
- Office hours will begin tomorrow, so please come ask us questions!
- Submit what you have at the end of lab to Gradescope.
 - Lab submissions are due at 6:00 pm.
 - The submission counts towards your lab grade.

Lab Groups

- You will now be assigned into your lab groups.
- Please sit with your group each week.
- Take the next 5 minutes to talk to each other.
- Introduce yourselves!
 - Name
 - Pronouns
 - What was one fun thing you did this summer?

Today's Goals

- Set up development environment
- Practice with higher-order functions

Writing and Running TypeScript

- Download the starter code from the course website (linked on Canvas).
- Unzip the folder and open it in VSCode.
- Run **"npm install"** in the same directory as the package.json file.
- Execute **"npm run test"** to run any existing tests.
- When you are ready to submit, run **"npm run build:submission"**
- Upload the resulting zip file to the corresponding assignment on Gradescope.

- If this process is new to you, your lab leaders can walk you through it!

Review of .map

```
// Sample Implementation
// `.map` is a method on Arrays
function map<T, U>(
  a: T[],
  f: (x: T) => U
): U[] {
  const result: U[] = [];
  for (let i = 0; i < a.length; ++i) {
    result.push(f(a[i]));
  }
  return result;
}
```

```
function double(x: number): number {
  return 2 * x;
}
```

```
const array = [1,2,3,4,5];
const newArray = array.map(double);
```

What is **newArray** ?

Reason about the code before typing and running it.

Review of `.filter`

```
// Sample Implementation
// `.filter` is a method on Arrays
function filter<T>(
  a: T[],
  f: (x: T) => boolean
): T[] {
  const result: T[] = [];
  for (let i = 0; i < a.length; ++i) {
    const x = a[i];
    if (f(x)) {
      result.push(x);
    }
  }
  return result;
}
```

```
function isEven(x: number): boolean {
  return x % 2 === 0;
}
const array = [1,2,3,4,5];
const newArray = array.filter(isEven);
```

What is `newArray` ?

Reason about the code before typing and running it.

Exercise 1: Higher-Order Functions

Write a function that takes in an array of number arrays and returns an array of number arrays where all negative values have been removed. Use higher-order functions (no loops or recursion).

```
[[1,-2,3], [0], [0,1,2,3], []]
```



```
[[1,3],[0], [0,1,2,3], []]
```

A number array is typed as `number[ ]`. An array of those is `number[ ][ ]`.

Exercise 1: Solution

```
function keepNonNegatives(a: number[]): number[] {  
  return a.filter(x => x >= 0);  
}  
  
function nonNegatives2D(arr: number[][]): number[][] {  
  return arr.map(keepNonNegatives);  
}
```

The helper function **keepNonNegatives** extracts the non-negative values from a 1D array. Because it takes in a **number[]**, it can be passed directly into **arr.map** in **nonNegatives2D**.

This is a good example of decomposing a problem into smaller problems.

Exercise 2: More Higher-Order Functions

Write a function that takes an array of string arrays (type `string[][]`) and returns a new array, which keeps only words longer than 3 characters, and then removes all empty rows. Use higher-order functions (no loops or recursion).

```
[[ "a", "line"], [ "longer", "words"],  
[ "for", "my", "car"]]
```



```
[[ "line"],  
[ "longer", "words"]]
```

(the last row would become empty, so it is removed)

Exercise 2: Solution

```
function longWords2D(text: string[][]): string[][] {  
    return text.map(row => row.filter(w => w.length > 3))  
                .filter(row => row.length > 0);  
}
```

The **.map** call produces a text with the same number of rows, filtering out short words from every row. Then, the **.filter** call produces a text that keeps only rows with nonzero length (word count).

We can't do **.filter(row => row !== [])**, since **[]** is a **new** array, and comparison works with references, so **!==** would always return true.

Final Thoughts

- Think carefully about your approach before starting to code.
- Start small and test often.
- Try to write reusable code and build upon it.
- This is a lot of new material, so come to office hours with any questions.
 - We would love to see you there! 😊
- It is okay if it takes you longer to finish the lab.
 - Keep working at it and try not to get discouraged.
 - Remember to submit!
- Try to get in the habit of reading the homework instructions as soon as they are released, and start working on the code sooner rather than later.